

𩚑

辶 穴 月 幺 長 言 馬 幺 長 丩 心

biáng, as in *biángbiáng* noodles · 58 strokes · eleven parts, every one of them common

Chinese character classification



- ◆ Input: **one glyph**, already cut out of the page.
- ◆ Output: **which character** it is, among thousands of classes.
- ◆ The reading step of OCR, for scripts with no alphabet.

Why it is hard

Same class, different looks:



Different classes, near-identical looks:

未 未 · 己 己 己 · 呆 杏

And far too many classes!

THE PROBLEM

A classifier cannot output a class it never saw

3,755

classes in the standard training set
(enough for daily reading)

97,000+

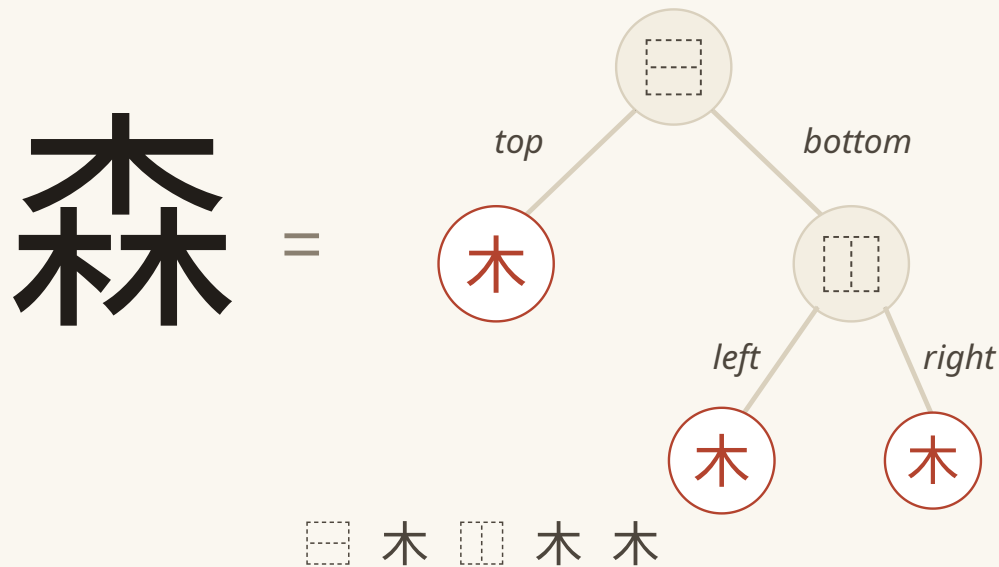
Han characters in Unicode
(old books use the long tail)

0 %

a softmax's accuracy
on everything else

*Zero-shot: recognise a character from its **description**, as you would read a new word by spelling it.*

A character is a tree of radicals



- ◆ ~500 radicals and 12 layout operators (left-right, top-bottom...) build every character.
- ◆ Unicode writes the tree as a string, the **IDS**. Every paper reads the same file.

The benchmark

Train on *m* handwritten classes (500 to 2,755). Test on **1,000 classes never seen**. Fewer training classes = harder.

Predict it, encode it, or align with it

1

Predict

Read the image, **write out** its tree, look it up in a dictionary.

≈ image captioning



next token, one at a time

2

Encode

Turn each tree into a **fixed vector**; train the image network to hit it.

≈ attribute-based zero-shot

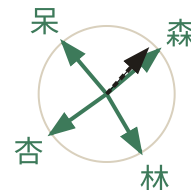


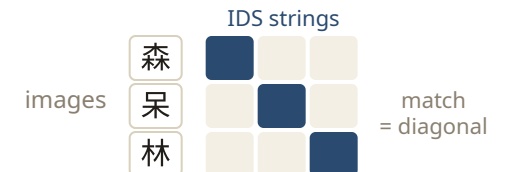
image
fixed codes

3

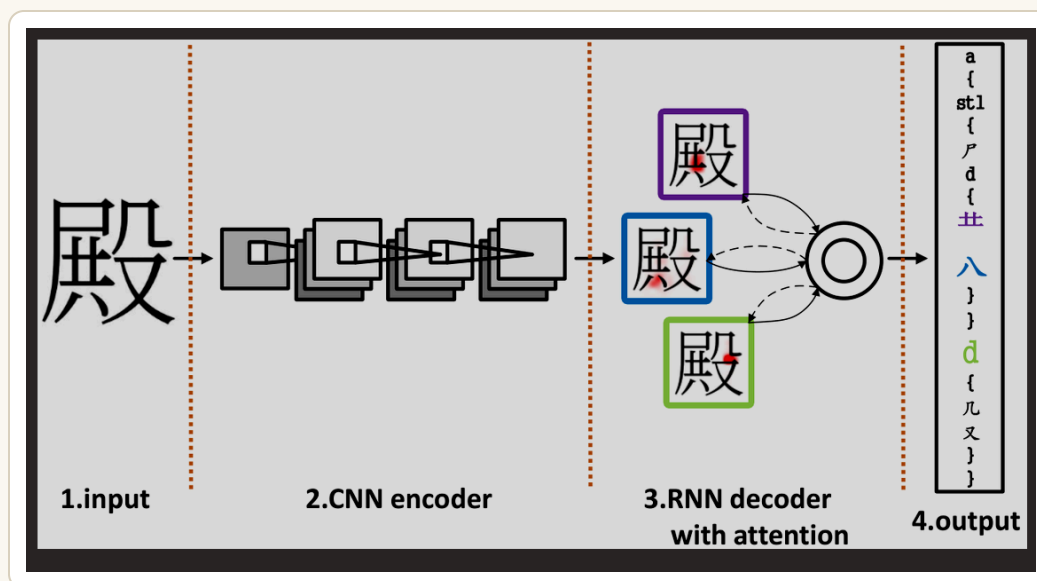
Align

Learn an encoder for the tree too, and match images to trees.

≈ CLIP



Write the tree, then look it up



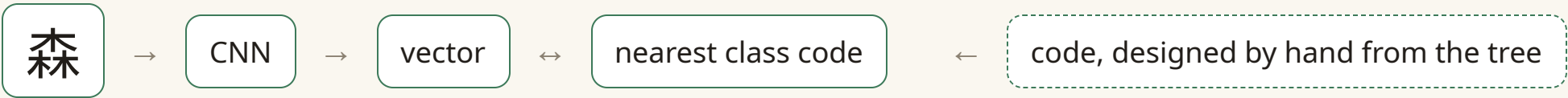
Each radical the decoder writes comes with an attention map: where it looked. RAN, Zhang et al., ICME 2018

RAN / DenseRAN 2018 → RSST 2024 → CDC-RAN 2025

- ◆ Interpretable: one radical, one region.
- ◆ Errors pile up along the sequence; one wrong token, wrong character.

The decoder learns to write **combinations it has seen**. Faced with a new character, it writes a familiar one.

Compile the tree into a fixed vector



Building a code: HDE (2020), simplified

$$\varphi(y) = \sum_{\text{radical } r} \alpha^{d_r} e_r + \lambda \sum_{\text{operator } s} \alpha^{d_s} e_s$$

one dimension per radical / operator; d = depth in the tree; $\alpha = \lambda = \frac{1}{2}$

森 = 𣏟 木 𣏟 木 木	depth	adds
𣏟	0	$\lambda \alpha^0 = 0.5$ to $e_{\text{𣏟}}$
木 (top)	1	$\alpha^1 = 0.5$ to $e_{\text{木}}$
𣏟	1	$\lambda \alpha^1 = 0.25$ to $e_{\text{𣏟}}$
木, 木	2	$2\alpha^2 = 0.5$ to $e_{\text{木}}$

$\Rightarrow \varphi(\text{森})$: 木 = **1.0**, 𣏟 = **0.5**, 𣏟 = **0.25**, every other dim 0.

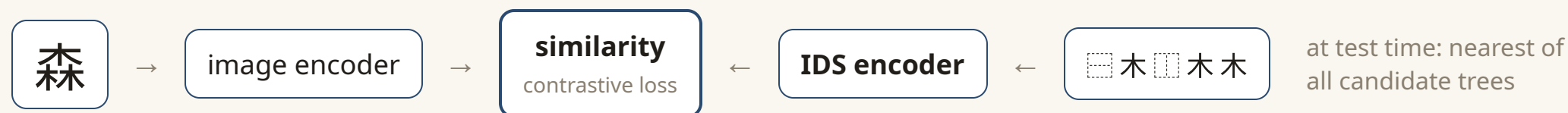
HDE 2020 → CUE 2023 → HierCode 2025 → JRED 2025

$$\hat{y} = \arg \max_{y \in \text{all classes}} \cos(f(x), \varphi(y))$$

train the image network f on seen classes; unseen ones only need their φ .

The code is **frozen before training**. 呆 (𣏟 over 木) and 杏 (木 over 𣏟) get the same counts: only a tiny position term, in the 4th decimal, tells them apart.

Learn both sides: CLIP, with the tree as caption



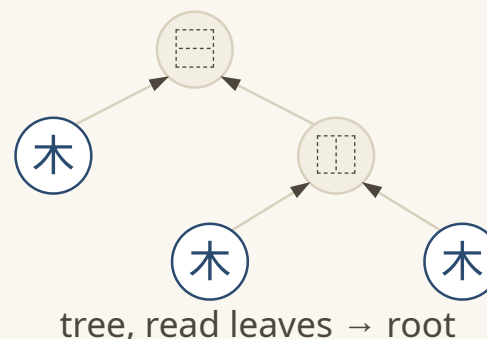
What moved: the IDS encoder. Same images, same strings, same benchmark ($m = 2,755$)



sequence · Transformer

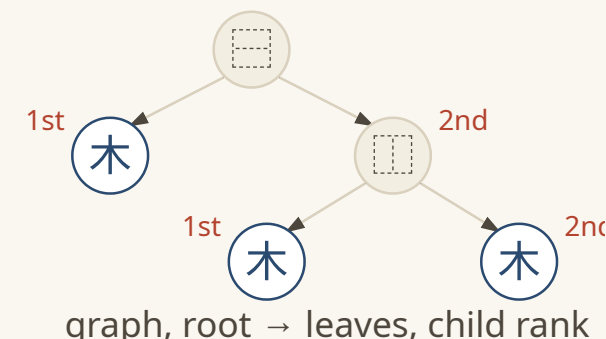
CCR-CLIP 2023 · 73.0

Structure written nowhere: learned from token order.



FT-CLIP 2025 · 79.0

each node attends to its children, and knows where they sit.



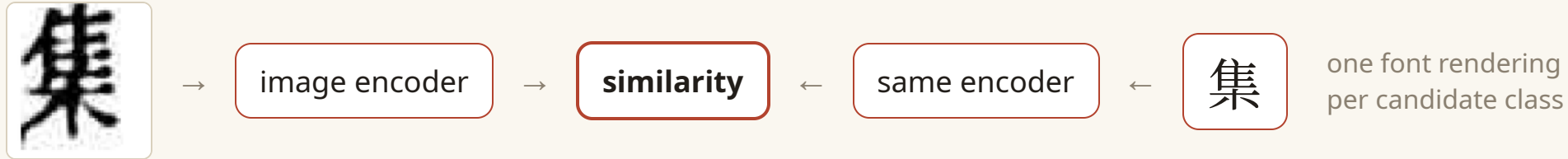
GRSTR 2026 · 80.6

a GNN pushes layout down, and 1st / 2nd child differ: 呆 ≠ 杏.

Next step, outside the encoder: match image regions to **each radical**, not one vector to one vector. **Hi-GITA 2025: 85.4.**

AND WITHOUT THE TREE?

Match the image to a printed font



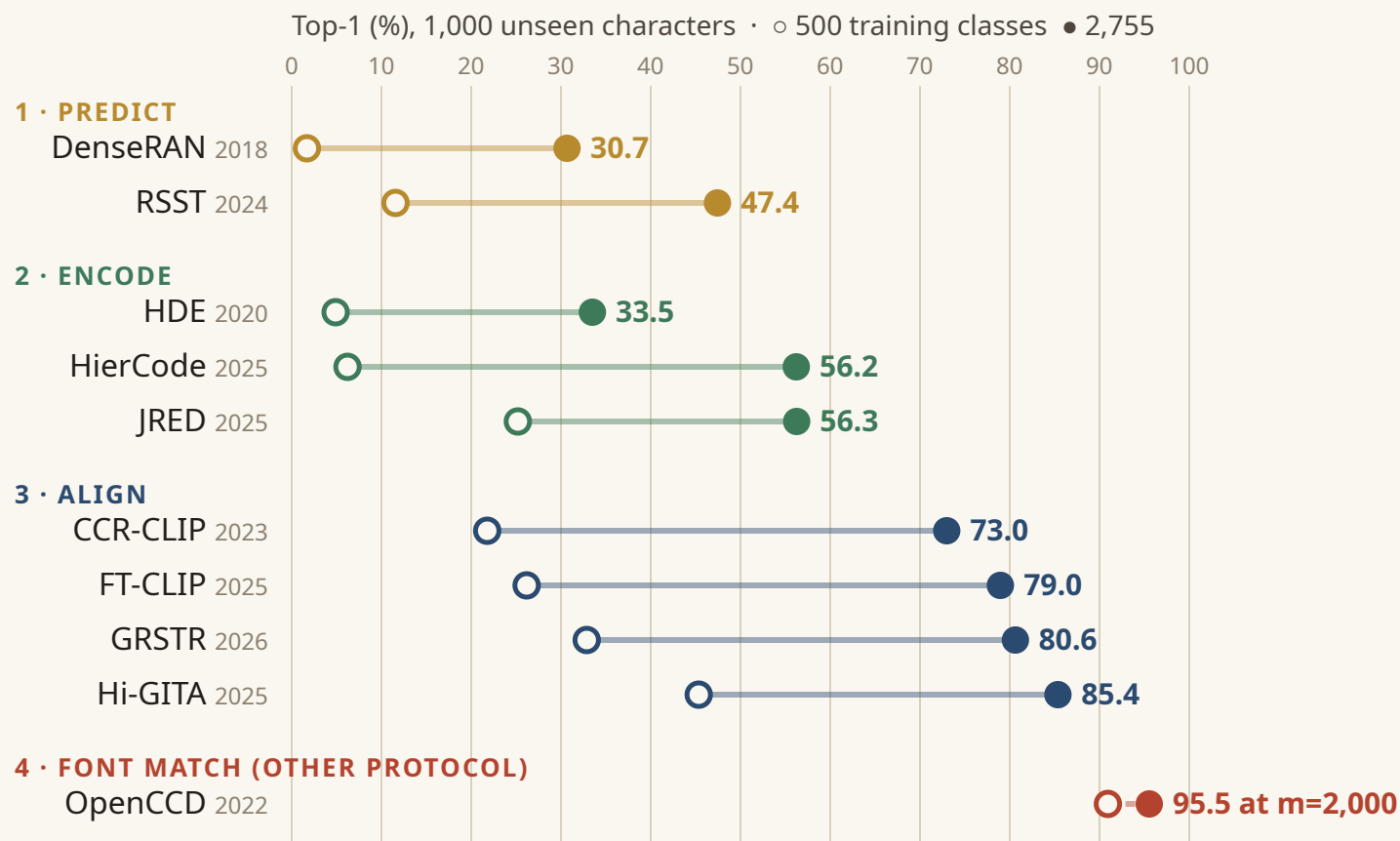
OpenCCD CVPR 2022 → PCSS 2024

- ◆ Same CLIP idea, but the "caption" is **a picture of the character in a font**, not its tree.
- ◆ Works where no tree exists: Japanese kana never seen in training, rare variants.

Needs a font that **draws every class**. Fewer than 5 fonts cover all 97k characters, and a modern font is not the shape of a 17th-century woodblock.

On the next chart it is drawn dashed: each unseen class comes with a printed glyph the tree methods never see, so it is not a like-for-like comparison.

Retrieval beats prediction, by a lot



+25 points. The first CLIP-style model beat six years of predictors.

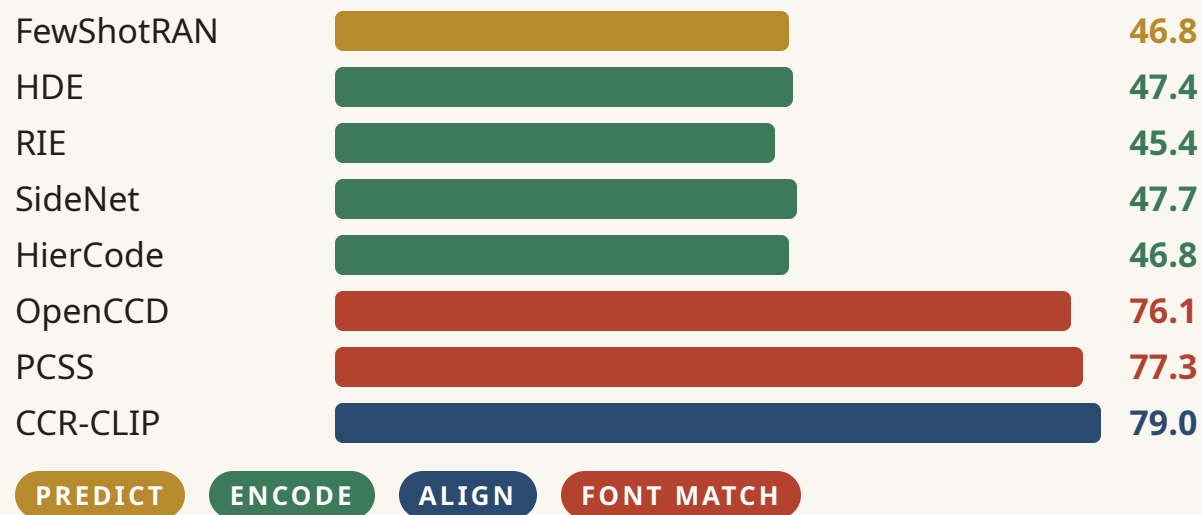
5,400× **faster.** Decoding: 1,666 ms per image. Retrieval: 0.31 ms.

10× How finely you split into radicals: **+27 pts.** A better backbone: **+2.7.**

HWDB → ICDAR 2013, Top-1. GRSTR, GL-HPN, Hi-GITA, OpenCCD tables. Dashed: given a printed glyph per unseen class.

97,455 classes: only the learned matchers hold up

Train on the 27,533 common characters → test on **69,922 never seen**. Top-1 (%).



≈47

Every fixed code lands at the same place, whatever its design. Many test characters contain radicals never seen in training.

79

CLIP on the tree still wins, +31 pts over the best code; font matching is close behind (76–77).

A learned embedding, used to fix a reader's mistakes

- ◆ I train a **contrastive glyph encoder**, family 3's recipe on the image side.
- ◆ Each glyph's nearest neighbours vote on its label: a **graph** over the whole book.

What the graph is for

1. **Detect errors:** a glyph whose reading disagrees with its look-alikes.
2. **Catch hallucinations:** the reader writes a plausible character that is not on the page.
3. **Expose model conventions:** one reader writes the Taiwan form, another the Hong Kong form, of the same glyph.

Run on a real Qing-dynasty volume: 770 pages, no ground truth.

■ CHAT = correction ■ CHAT = original ■ CHAT = neither							
梅 梅 → 梅 CHAT: 梅	山 江 → 山 CHAT: 山	詩 山 → 詩 CHAT: 詩	藥 痕 → 藥 CHAT: 藥	載 東 → 載 CHAT: 載	贈 農 → 贈 CHAT: —	得 太 → 得 CHAT: —	添 萬 → 添 CHAT: 添
輓 風 → 輓 CHAT: 輓	一 二 → 一 CHAT: 二	稿 山 → 稿 CHAT: 稿	稿 山 → 稿 CHAT: 稿	霞 雲 → 霞 CHAT: 雪	橫 縱 → 橫 CHAT: 橫	集 堂 → 集 CHAT: 集	集 詩 → 集 CHAT: 集
州 璃 → 州 CHAT: 慨	露 路 → 露 CHAT: 路	二 一 → 二 CHAT: —	一 三 → 一 CHAT: —	請 調 → 請 CHAT: 師	跋 時 → 跋 CHAT: 跋	浪 跋 → 浪 CHAT: 浪	已 己 → 已 CHAT: 已

Corrections kept on volume 611. Blue: a second, independent reader agrees.

謝
謝

Thank you for your attention

Questions?